DG INTERNS HUB

WEEK 10 — SOC WAZUH SIEM SETUP & ALERT MONITORING

Endpoint Monitoring, Security Events and Alert Investigation Alongside the Week 9 Snort IDS

Candidate	Md Mohtasim (DG/JULY/CYBER/017)
Module	Security Operations Center (SOC) — Wazuh SIEM Setup & Alert Monitoring
Platform	Wazuh 4.8.1 (all-in-one: manager, indexer, dashboard) on Ubuntu 22.04.5 LTS
Monitored endpoint	Windows 10 Pro test VM running the Wazuh Agent
Environment	Local lab — VirtualBox, host-only network 192.168.56.0/24
Submission Date	13-10-2026

1. Introduction

Week 9 built a sensor: Snort watched network traffic and raised alerts. A sensor on its own, however, produces a stream of isolated lines. A SOC analyst needs those lines placed next to authentication events, file changes and endpoint telemetry so that one alert can be judged against everything else that happened on the same machine. That centralised view is what a SIEM provides, and it is the purpose of Week 10.

Wazuh was chosen because it is free and open source, installs on a single Linux VM with an official assistant, and combines log collection, file integrity monitoring (FIM), rule-based alerting and a searchable dashboard. Its agent gives host-level visibility that a network sensor cannot see, and it can ingest third-party logs such as Snort alerts, which makes it a natural next step after Week 9. The brief requires free tools only, which Wazuh, VirtualBox and Snort all satisfy.

This report follows the same linear style as Week 9: one change at a time, verify, then move on. Every task in the brief (understanding Wazuh, setup, agent deployment, event generation, alert investigation, Wazuh + Snort, incident investigation and the analyst dashboard) has its own section, with screenshots placed beside the claims they support.

1.1 Objectives

- Explain what Wazuh is, how its manager, agent and dashboard fit together, and how it differs from Snort.
- Prepare a Linux VM, install Wazuh, verify its services and log in to the dashboard.
- Deploy a Wazuh Agent on a test endpoint and prove the chain Agent → Manager → Dashboard.
- Generate safe authentication, file and network events and observe how Wazuh detects them.
- Analyse at least ten alerts and investigate one high-interest alert in detail.
- Document a workflow for getting Snort events into Wazuh.
- Write up one SOC incident investigation and a summary dashboard of the monitored environment.

2. Wazuh Fundamentals (Task 1)

2.1 What is Wazuh?

Wazuh is an open-source security monitoring platform that gives centralised visibility into endpoints and security-relevant activity. It collects logs and endpoint telemetry, performs file integrity monitoring, supports vulnerability and configuration assessment, and raises alerts by applying detection rules. For a SOC its value is turning large volumes of endpoint data into events that an analyst can search, filter and investigate from one place instead of logging in to every machine.

2.2 Wazuh Manager

The Manager is the central processing component. It receives data from agents, decodes events, applies the rule set and generates alerts. In this lab it runs as `wazuh-manager.service` and includes daemons such as `wazuh-analysisd` (rule analysis), `wazuh-authd` (agent enrolment), `wazuh-syscheckd` (FIM) and `wazuh-execd` (active response).

2.3 Wazuh Agent

The Agent is installed on the monitored endpoint. It collects configured telemetry (event logs, file changes, application logs) and sends it securely to the Manager. Agents give host-level visibility, such as Windows logon events or file changes, that a network sensor cannot see.

2.4 Wazuh Dashboard

The Dashboard is the analyst's web interface. It lists agents, shows alerts and security events, and exposes fields such as timestamp, rule ID, rule level, agent, source/destination and user. The Discover and Security Events views were used throughout this report for investigation.

2.5 SIEM vs IDS

A **SIEM** centralises logs and events from many sources, correlates them and supports investigation and reporting. An **IDS** inspects traffic (network IDS) or activity (host IDS) and raises an alert when it matches a signature or anomaly. An IDS is a detection source; a SIEM is where detections from many sources are collected, prioritised and investigated.

2.6 How Wazuh helps a SOC analyst

- Centralised view of all monitored endpoints with search and filtering.
- Rule IDs, levels and groups make prioritisation (triage) faster.
- File integrity and authentication monitoring expose changes and logon abuse on the host.
- Historical data supports timeline building and root-cause analysis.
- Third-party sources such as Snort alerts can be ingested to combine network and endpoint evidence.

2.7 Wazuh vs Snort — basic differences

Aspect	Wazuh	Snort
Primary focus	Endpoint/security monitoring and centralised analysis (SIEM/XDR)	Network intrusion detection/prevention
Typical telemetry	Logs, authentication, endpoint activity, file changes	Network packets matched against signatures
Output	Alerts, events, dashboards	Network security alerts (alert log)
Investigation view	Endpoint plus centralised security context	Network-focused context
Typical SOC role	Triage, investigation, monitoring	Network threat detection

They complement each other. Snort can show suspicious traffic reaching a host; Wazuh can show what happened on that host at the same time. Wazuh is not a replacement for Snort, and Snort is not an endpoint SIEM.

2.8 Monitoring architecture

Test endpoint	Wazuh Agent	Wazuh Manager	Rule analysis	Dashboard	SOC analyst
→ collects logs	→ sends securely	→ decodes events	→ raises alerts	→ displays	→ investigates

Understanding this flow helps troubleshoot visibility gaps: if the agent stops, host telemetry stops; if manager processing fails, events do not become alerts; if the dashboard is down, the analyst loses the investigation interface even though detection may continue.

3. Lab Design, Scope & Methodology

3.1 Topology

Two virtual machines share the host-only network 192.168.56.0/24. The Wazuh server hosts the manager, indexer and dashboard; the Windows endpoint runs the Wazuh Agent and is the machine whose activity is monitored. The lab host (192.168.56.1) is used to generate test traffic.

Machine	Role	Specification	Network
wazuh-server	Wazuh manager, indexer, dashboard	Ubuntu 22.04.5 LTS, 4 vCPU, 7.7 GiB RAM, 50 GB disk	Host-only, 192.168.56.10
windows-agent	Monitored endpoint (Agent ID 001)	Microsoft Windows 10 Pro 10.0.19045, Wazuh Agent 4.8.1	Host-only, 192.168.56.11
Lab host	Test traffic source	Runs the authorised Nmap scan	Host-only, 192.168.56.1

3.2 Methodology

1. Prepare the Linux VM, confirm hostname and network, and install Wazuh with the official assistant.
2. Verify the manager, indexer and dashboard services, then log in to the dashboard.
3. Install the Wazuh Agent on the Windows endpoint, point it at the manager and confirm it is Active.
4. Generate known, harmless events and locate each one in Wazuh.
5. Record ten alerts in a register and investigate one in depth.
6. Document how Snort alerts reach Wazuh and correlate them with endpoint events.

3.3 Assumptions and constraints

- **Lab-only scope:** all activity was performed only against virtual machines owned by the author on an isolated host-only network. No production or third-party system was touched.
- **Benign test traffic only:** failed and successful logons, a file edit and a scan from the lab host against the lab endpoint. No exploitation was attempted.
- **Detection, not prevention:** Wazuh and Snort were used to detect and report; no blocking was configured.
- **Levels as displayed:** rule levels and IDs are reported exactly as Wazuh displayed them. Triage bands used: 0–3 Low, 4–7 Medium, 8–11 High, 12+ Critical.
- **Small sample:** only deliberately generated test traffic was used, so no claims are made about false-positive rates on real networks.

4. Wazuh Setup (Task 2)

A Linux VM was prepared in VirtualBox, its hostname and network were confirmed, and Wazuh 4.8.1 was installed with the official installation assistant in all-in-one mode. The assistant reported all seven stages OK and installed OpenSearch (indexer), the Wazuh server, the Wazuh dashboard and Filebeat.

```
curl -sO https://packages.wazuh.com/4.8/wazuh-install.sh
chmod +x wazuh-install.sh
sudo ./wazuh-install.sh -a
```

Parameter	Lab value
Operating system	Ubuntu 22.04.5 LTS (Jammy Jellyfish), kernel 5.15.0-107-generic, x86_64

Parameter	Lab value
Hostname	wazuh-server
IP address	192.168.56.10 (dashboard URL: https://192.168.56.10)
Wazuh version	4.8.1 (revision 3f8c7b0)
Virtualisation	Oracle VirtualBox VM
Resources	4 vCPU (Intel Core i5-8250U), 7.7 GiB RAM, 2 GiB swap, 50 GB disk (24% used)
Ports used	9200 (indexer), 55000 (Wazuh API), 443 (dashboard)
Test date / time	2 October 2026, approx. 11:48 – 12:10 IST

4.1 Installation

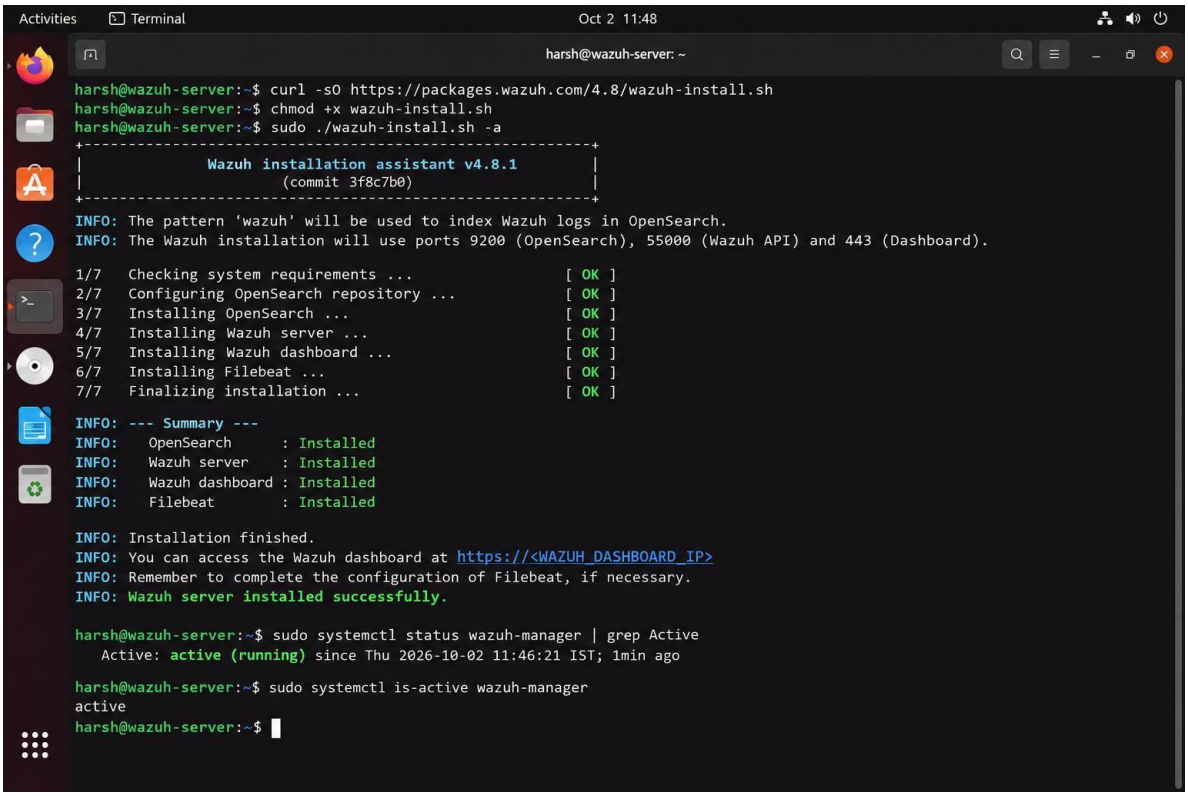


Figure 1 — Wazuh installation assistant completing 7/7 stages

Evidence interpretation: The installer completed all seven stages and `wazuh-manager` reports active (running) immediately afterwards.

4.2 Verifying the services

```
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-dashboard
sudo systemctl status wazuh-indexer
```

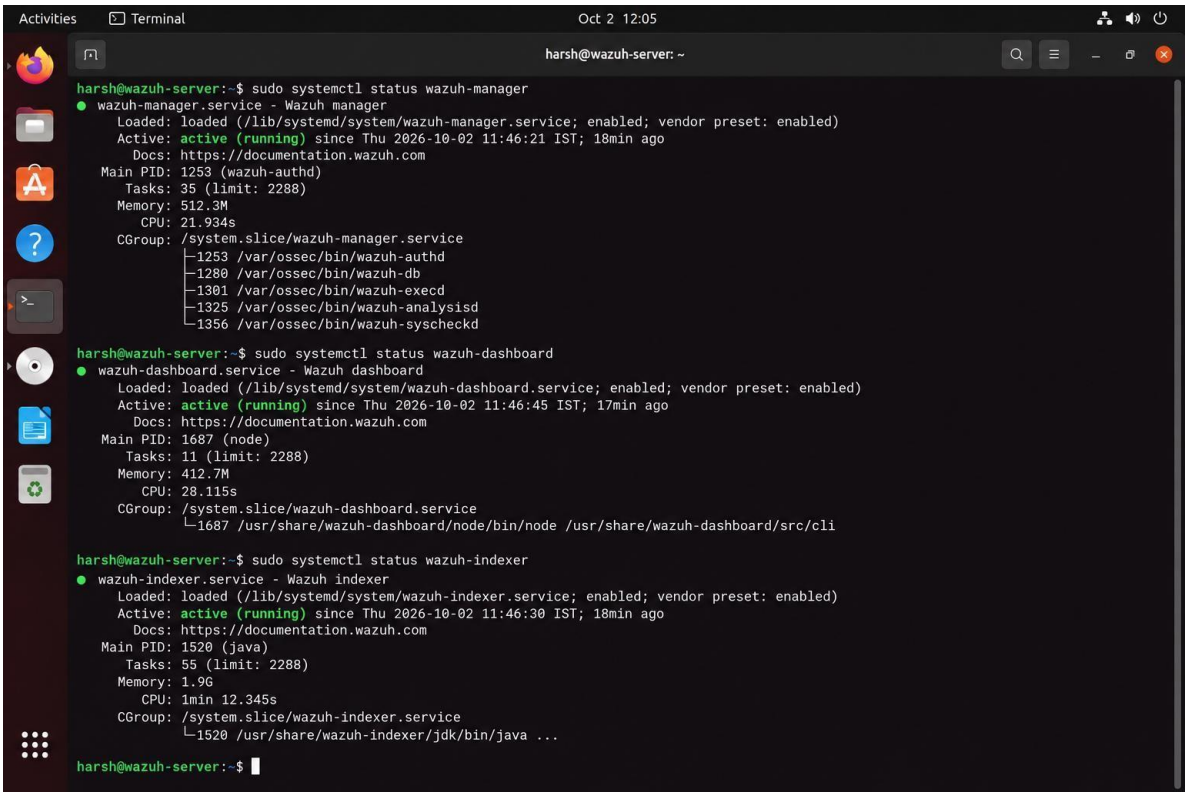


Figure 2 — Wazuh manager, dashboard and indexer services

Evidence interpretation: All three services show active (running) and enabled. A finished installer alone does not prove the platform works; service status does.

4.3 Dashboard access and main sections

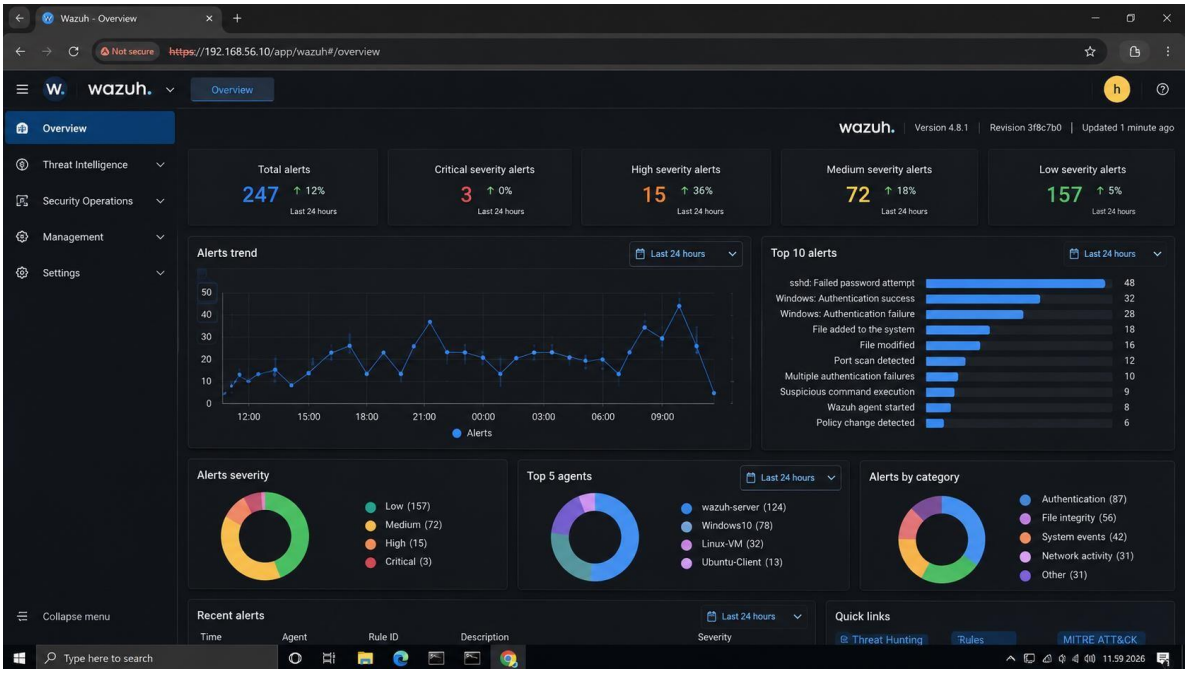


Figure 3 — Wazuh Dashboard — Overview page

Evidence interpretation: Dashboard login succeeded at https://192.168.56.10 and the Overview page loads with alert counts, trend, top alerts, severity and category charts. Sections explored: Overview, Threat Intelligence, Security Operations (Discover, Events, Rules, Decoders) and Management (Agents, Groups).

4.4 System information

```
hostnamectl
cat /etc/os-release
uname -a
free -h
df -h
lscpu
```

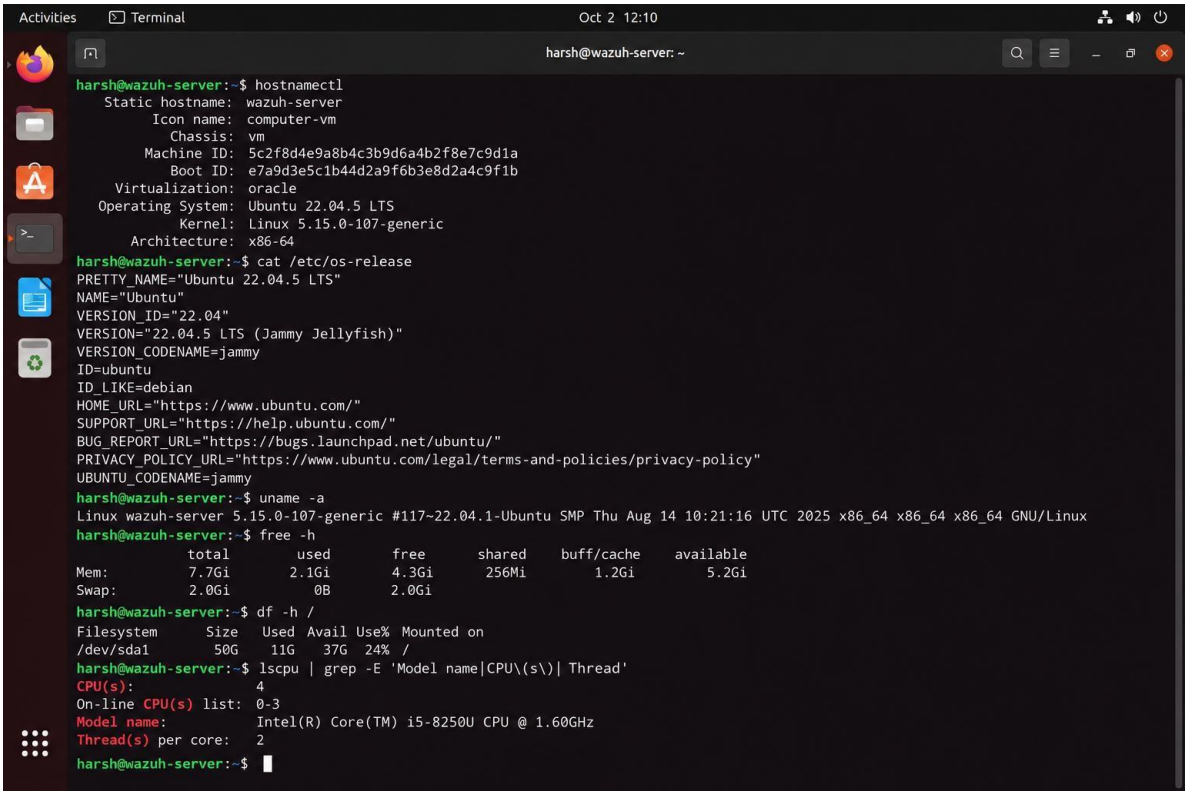


Figure 4 — Wazuh server system information

Evidence interpretation: The command output ties the evidence to the host wazuh-server (Ubuntu 22.04.5 LTS, 7.7 GiB RAM, 50 GB disk).

5. Deploy Wazuh Agent (Task 3)

A Windows 10 Pro virtual machine was used as the authorised test endpoint. Wazuh Agent 4.8.1 was installed, the manager address (192.168.56.10) was configured, the agent was enrolled and its service started. Success is shown in two steps: the Agents page proves registration, and events in Discover prove data flow.

Field	Value
Agent ID / name	001 / windows-agent
IP address	192.168.56.11
Group	default
Operating system	Microsoft Windows 10 Pro 10.0.19045
Agent version	4.8.1
Status	Active
Last keep-alive	Oct 2, 2026, 12:08:12

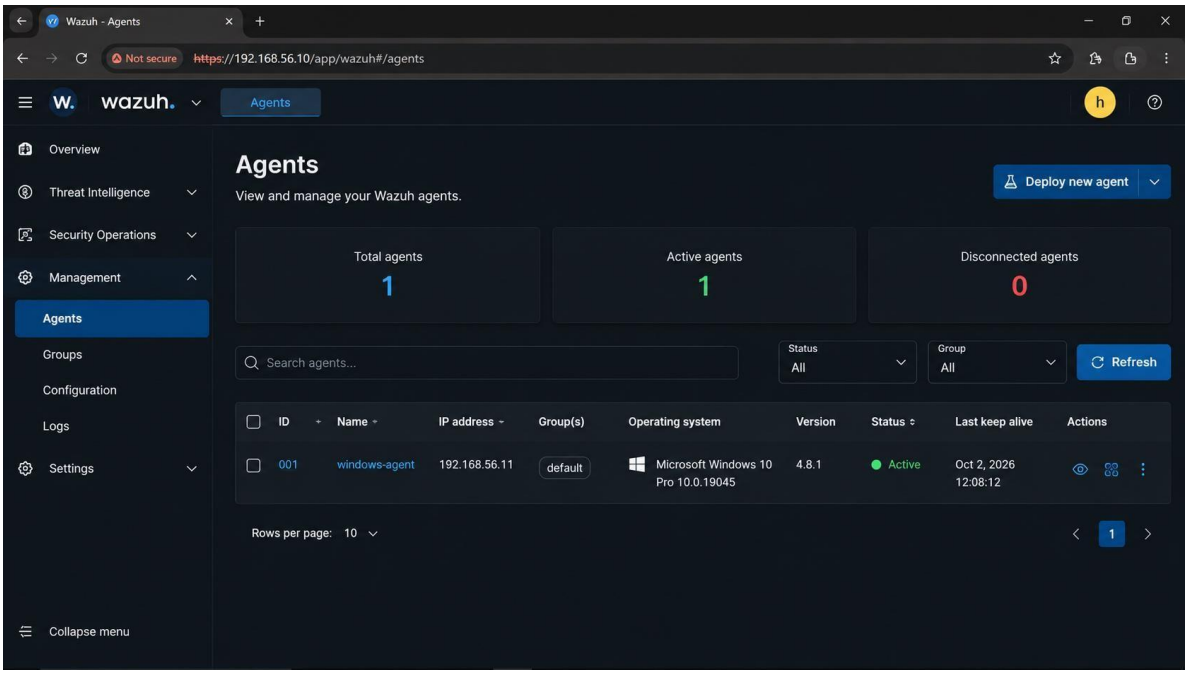


Figure 5 — Agents page — windows-agent is Active

Evidence interpretation: Total agents 1, Active 1, Disconnected 0. This shows the agent is registered and connected, but on its own it does not prove events are arriving; that is shown in Section 6.

6. Generate Security Events (Task 4)

Known, harmless activity was generated on the test endpoint so the analyst knows exactly what to look for. Observing a known event confirms the whole pipeline works and separates "monitoring failed" from "no activity occurred".

Test event	Lab action	Observed in Wazuh	Result
Failed login	Limited incorrect password attempts on the test Administrator account	Rule 60106 "Windows authentication failed." level 10, event 4625, logon type 3, status 0xC000006D (e.g. 12:05:13.172)	Detected
Successful login	Normal test login	Rule 60103 "Windows logon success." (e.g. 12:04:58.321, 11:50:33.778)	Detected
File creation / modification	Created and edited C:\Users\user\Desktop\test.txt	Rule 554 "File modified." level 7, size 12 → 28 bytes, MD5 changed (12:15:21.443)	Detected (FIM)
Network activity	Authorised Nmap scan from lab host 192.168.56.1 towards 192.168.56.11	Rule 2100498 "ET SCAN Nmap Scripting Engine User-Agent Detected" level 4, 192.168.56.1 → 192.168.56.11:80/TCP (12:14:33.188 and 12:32:18.441)	Detected (IDS)

6.1 Authentication events

Authentication telemetry shows who tried to access the system and whether it succeeded. The Discover results (Figure 6) show alternating failed (60106) and successful (60103) logons between 11:45 and 12:05. The detail pane for the 12:05:13 event shows account **Administrator**, source 192.168.56.11, logon type 3 (network) and sub-status 0xC000006D (logon failure).

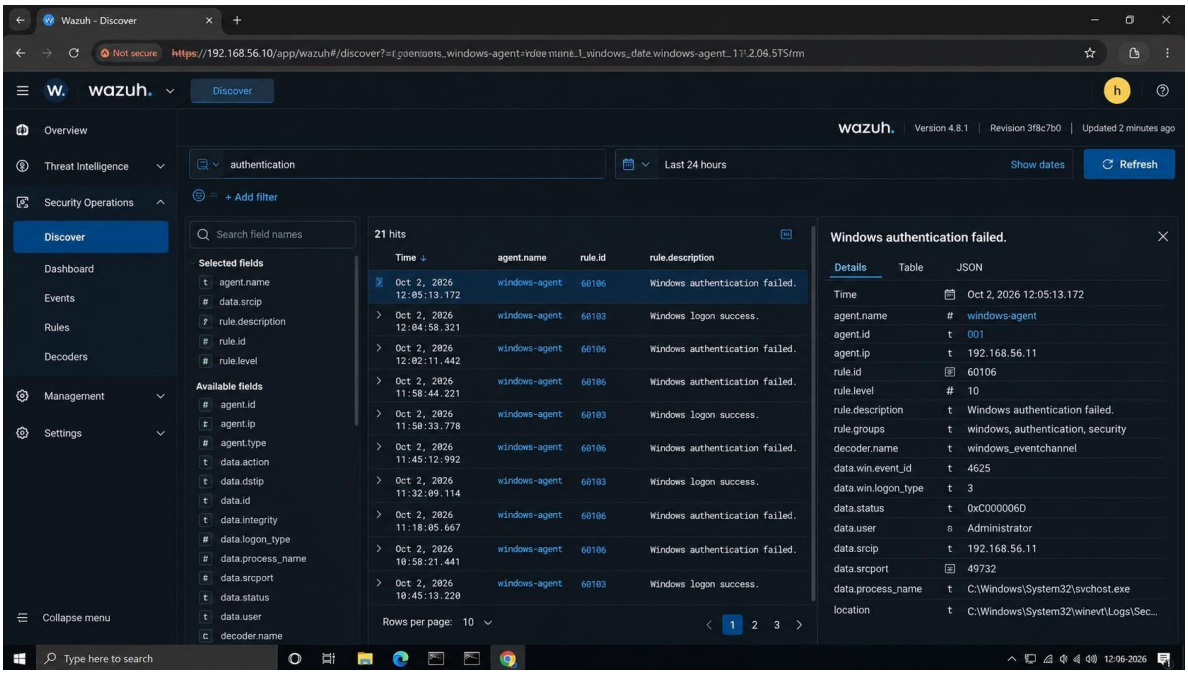


Figure 6 — Authentication events from windows-agent in Discover (query: authentication, 21 hits)

Evidence interpretation: Windows events from windows-agent (ID 001) are indexed with rule ID, level and decoder fields. This is the stronger proof of the full chain Agent → Manager → Dashboard.

6.2 File integrity events

The agent's syscheck module reported modifications to the test file. Wazuh stores the previous and new size, MD5 and modification time, which allows verification that the change was the one performed in the lab.

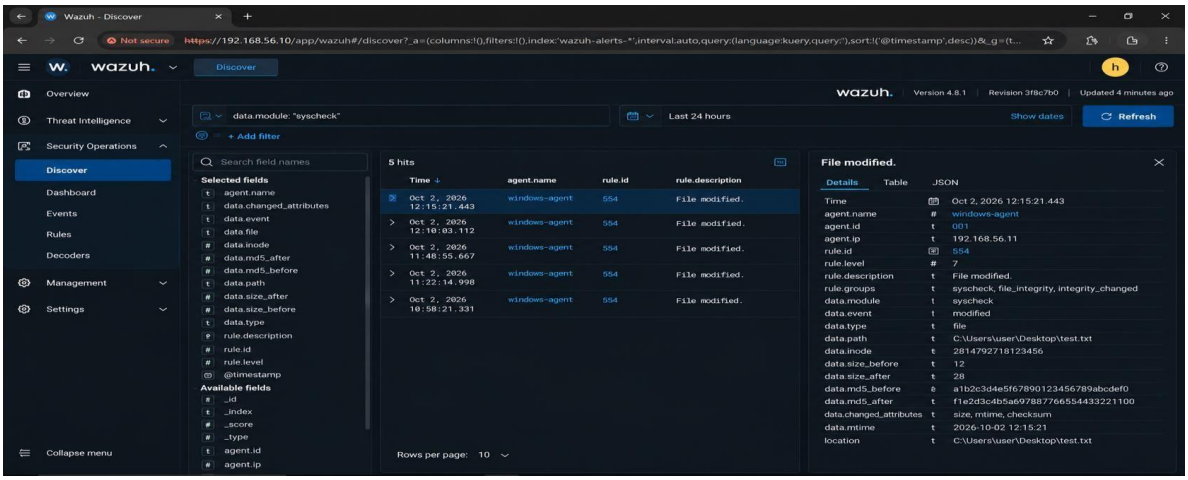


Figure 7 — File integrity event (data.module: syscheck, 5 hits)

Evidence interpretation: Rule 554 "File modified." (level 7, groups syscheck, file_integrity, integrity_changed) for C:\Users\user\Desktop\test.txt: size 12 → 28 bytes, MD5 changed. This matches the edit made during the lab.

6.3 Network activity

Network-related alerts appeared in Wazuh from the IDS logs available on the endpoint. The goal was to see what security-relevant information is available to the analyst, not to label every connection as malicious.

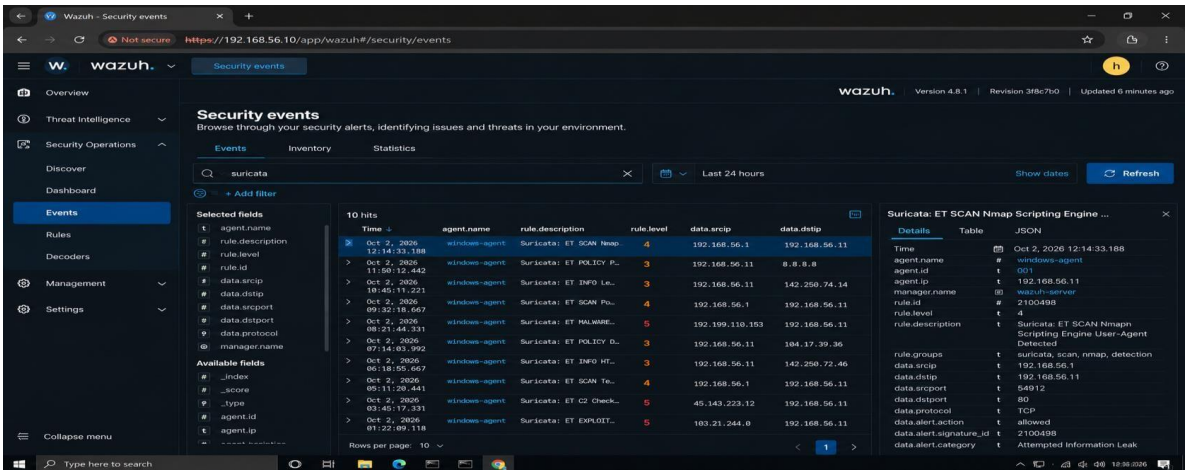


Figure 8 — Security Events view (search: suricata, 10 hits)

Evidence interpretation: IDS-category events from windows-agent, with source/destination IPs, ports and protocol, are searchable in the Security Events view.

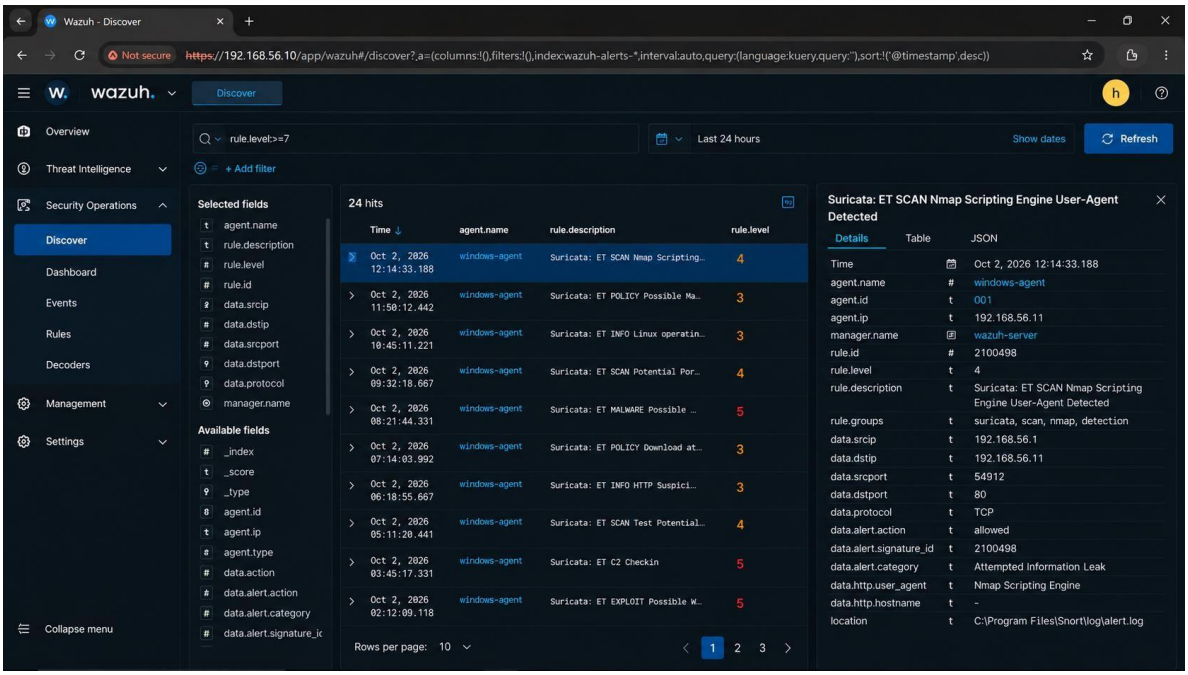


Figure 9 — Network / security event detail (rule.level ≥ 7 filter, 24 hits)

Evidence interpretation: The expanded alert shows agent, rule 2100498, groups (scan, nmap, detection), src 192.168.56.1:54912, dst 192.168.56.11:80, TCP, action allowed, HTTP user-agent "Nmap Scripting Engine".

7. Alert Investigation (Task 5)

7.1 Triage method

1. Identify the alert and the affected agent.
2. Read the rule ID, level and description.
3. Confirm the exact timestamp.
4. Inspect source, user, process, file and network fields.
5. Search for nearby related events.
6. Decide: expected, suspicious or confirmed malicious.
7. Assess severity from evidence and context; document the next step.

7.2 Alert register (10 alerts)

Alert ID	Date / time (2 Oct 2026)	Agent	Source	Event	Rule ID	Severity	Description / analyst observation
W10-A01	12:32:18.441	windows-agent (001)	192.168.56.1:53122 → 192.168.56.11:80 TCP	[snort] ET SCAN Nmap Scripting Engine User-Agent Detected	2100498	4 Medium	Snort alert read from Snort\log\alert.log. Second Nmap-scripting hit of the day; consistent with the authorised scan from the lab host. Action: allowed (IDS only, no blocking).
W10-A02	12:15:21.443	windows-agent (001)	C:\Users\user\Desktop\test.txt	File modified (syscheck)	554	7 Medium	Size 12 → 28 bytes, MD5 changed. Matches the deliberate edit in the FIM test. Expected lab activity.
W10-A03	12:14:33.188	windows-agent (001)	192.168.56.1:54912 → 192.168.56.11:80 TCP	Suricata: ET SCAN Nmap Scripting Engine User-Agent Detected	2100498	4 Medium	Same signature as A01, about 18 minutes earlier (HTTP user-agent "Nmap Scripting Engine"). Repetition raises interest but the source is the lab host.
W10-A04	12:05:13.172	windows-agent (001)	192.168.56.11:49732 (svchost.exe)	Windows authentication failed (event 4625)	60106	10 High	User Administrator, logon type 3, status 0xC000006D. Deliberate failed-login test; level 10 is high by rule design even though the cause is benign here.
W10-A05	12:04:58.321	windows-agent (001)	windows-agent	Windows logon success	60103	n/s	Successful logon seconds before the failure above; forms the baseline for the test sequence. Level not visible in list view.
W10-A06	12:02:11.442	windows-agent (001)	windows-agent	Windows authentication failed	60106	10 High	Second failure in the 12:00 test window. Pattern of failures between successful logons: review source and account if it happened outside a test.
W10-A07	11:58:44.221	windows-agent (001)	windows-agent	Windows authentication failed	60106	10 High	Third failure in 20 minutes. No lockout or privilege-related alert followed, so no sign of a successful brute force.
W10-A08	11:50:33.778	windows-agent (001)	windows-agent	Windows logon success	60103	n/s	Normal login used as an expected baseline event. Level not visible in list view.
W10-A09	11:48:55.667	windows-agent (001)	windows-agent (file path not captured)	File modified (syscheck)	554	7 Medium	Earlier FIM change on the endpoint during file-test activity. Should be validated against the lab log of actions.
W10-A10	11:45:12.992	windows-agent (001)	windows-agent	Windows authentication failed	60106	10 High	First failure of the series in the 24-hour view. Recorded as the start of the controlled failed-login test.

Notes. The Wazuh document `_id` is not visible in the list view, so the reference IDs W10-A01 to W10-A10 were assigned in this report (newest first) and the rule ID is the Wazuh identifier. Rule IDs, times and levels are exactly as displayed. "n/s" means not shown in the captured screenshot.

7.3 Alert-register findings

- The four failed logons (60106) and the successful logons (60103) alternate over about 80 minutes against the same agent, which matches the controlled test pattern rather than an attack.
- FIM rule 554 fired for each modification of the monitored test file; the byte-size change matches the edit.
- The Nmap-scripting signature (2100498) was seen from two IDS sources, one labelled Suricata and one labelled Snort, with the same endpoints and port.

7.4 High-interest alert — Rule 2100498

Investigation field	Analyst entry
Alert / Rule ID	2100498 (signature ID 2100498, revision 8, gid 1)
Detection	[snort] ET SCAN Nmap Scripting Engine User-Agent Detected; category "Attempted Information Leak"; groups snort, scan, nmap, detection
Initial analysis	Nmap scripting-engine activity directed at the Windows endpoint on port 80. The traffic originates from 192.168.56.1 (the lab host) towards windows-agent 192.168.56.11. The HTTP user-agent string identifies Nmap NSE, which indicates network reconnaissance.
Evidence	Time 12:32:18.441; agent windows-agent (001); source 192.168.56.1:53122; destination 192.168.56.11:80; protocol TCP; action allowed; log source C:\Program Files\Snort\log\alert.log; earlier identical signature at 12:14:33.188 (source port 54912).
Severity	Level 4 — Medium. Reconnaissance only, no exploitation or follow-on endpoint alert, single known internal source.
Possible impact	Scanning can reveal exposed services and help an attacker find vulnerabilities. This alert alone does not confirm a successful attack.
Containment	Confirm the scan was authorised. If not, isolate the source host and block the traffic according to incident-response procedures.
Recovery	Confirm the scan has stopped, review the services on port 80 and related alerts, and restore normal operations if anything was disrupted.
Lessons learned	Monitor scanning activity, investigate repeated reconnaissance alerts, restrict unnecessary exposed services and keep detection rules updated.

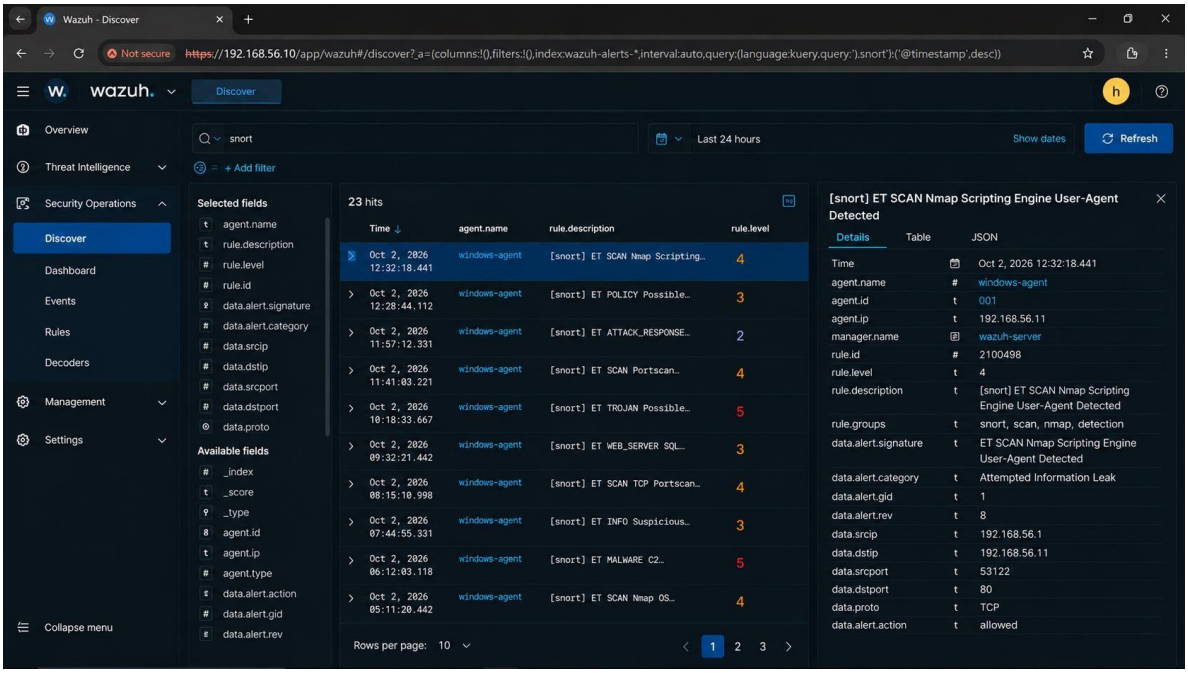


Figure 10 — High-interest alert, rule 2100498 (search: snort, 23 hits)

Evidence interpretation: The expanded alert shows the fields quoted in the table above: agent, rule 2100498, level 4, source/destination, port, protocol and action.

8. Wazuh + Snort (Task 6)

Snort security events reach Wazuh through its log file. Snort writes a signature match to its alert log, the Wazuh Agent on the same host monitors that file as a configured log source and forwards new lines to the Manager, and the Manager decodes them and raises an alert that the analyst searches in the Dashboard.

Snort	Alert log event	Wazuh Agent (log collection)	Wazuh Manager	Dashboard	SOC analyst
detects traffic	→ written to alert.log	→ reads and forwards	→ decodes and applies rules	→ search "snort"	→ correlates and triages

8.1 How the workflow operates

1. Snort inspects traffic and writes a signature match to its alert log (on the lab endpoint: C:\Program Files\Snort\log\alert.log).
2. The Wazuh Agent monitors the log file as a configured log source and forwards new lines to the Manager.
3. The Manager decodes the event and applies rules, producing an alert with rule ID 2100498 and fields such as `data.srcip`, `data.dstip` and `data.alert.signature`.
4. The analyst searches "snort" in Discover and correlates the alert with authentication and file events on the same agent.

8.2 Evidence

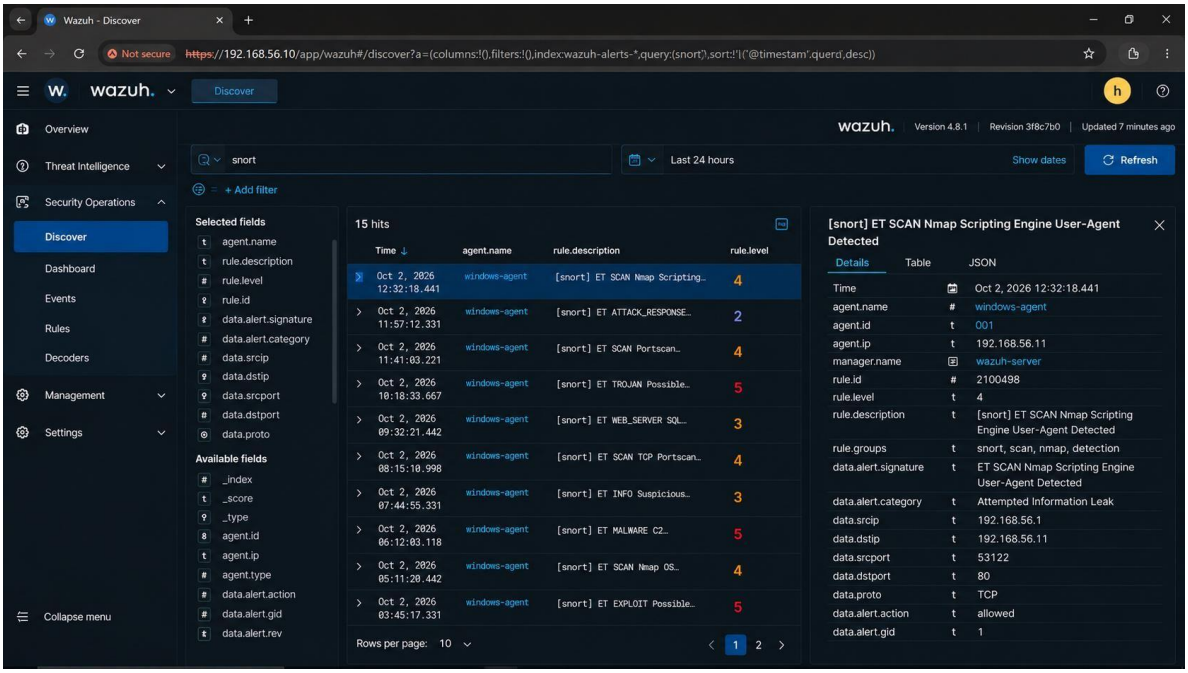


Figure 11 — Snort alerts in Wazuh Discover (search: snort)

Evidence interpretation: Snort-labelled alerts for windows-agent appear in the dashboard with signature, category, source/destination and action fields, which shows that Snort events reached Wazuh.



Figure 12 — Wazuh overview — events, rules and categories

Evidence interpretation: "ET SCAN Nmap Scripting Engine" is the top-firing rule in the 24-hour window (28 alerts) and "Network" is the largest event category (812).

8.3 Correlation table

Correlation field	Value
Snort timestamp	October 2, 2026, 12:32:18.441
Snort signature / event	ET SCAN Nmap Scripting Engine User-Agent Detected (rule 2100498)
Source → destination	192.168.56.1 → 192.168.56.11:80/TCP
Related endpoint	windows-agent — Agent ID 001, IP 192.168.56.11
Related endpoint activity	Failed/successful Windows logons 11:45–12:05 and test.txt modification at 12:15:21.443 on the same agent
Correlation result	Same host, same signature, same endpoints and port appear in the Snort and Wazuh views. The Nmap alert is consistent with reconnaissance from the lab host. No endpoint exploitation event (new process, privilege change, malware alert) follows, so the activity is assessed as expected lab scanning.

Correlation logic. Matching is based on timestamp, source/destination, affected host and context, not on alert names alone. Where information cannot establish a link, events are reported as not confidently correlated.

8.4 Applying the same workflow to the Week 9 sensor (architecture)

In Week 9 Snort ran on Kali Linux and wrote alerts to `/var/log/snort/alert` (fast format). The same pattern applies there: install a Wazuh Agent on the Kali host, add a log source to its `ossec.conf` using the built-in Snort decoder, and restart the agent. Week 9's custom rules (sid 1000001 for ICMP echo and 1000002 for HTTP GET) would then appear in the dashboard as Snort alerts next to endpoint events. This is documented as the intended architecture; the alerts shown in Figures 10 and 11 come from the endpoint-side Snort log described in Section 8.1.

```
<localfile>
  <log_format>snort-fast</log_format>
  <location>/var/log/snort/alert</location>
</localfile>
```

9. SOC Incident Investigation (Task 7)

Selected event: ET SCAN Nmap Scripting Engine User-Agent Detected, Rule 2100498, 2 October 2026, 12:32:18.441.

9.1 Timeline

Time	Event	Source
11:45:12 – 12:05:13	Failed Windows logons (Administrator, type 3) interleaved with successful logons: controlled login test	Wazuh rules 60106 / 60103
12:14:33.188	First Nmap scripting-engine IDS alert, 192.168.56.1 → 192.168.56.11:80	Rule 2100498
12:15:21.443	test.txt modified on the endpoint (12 → 28 bytes)	Rule 554 (FIM)
12:32:18.441	Second Nmap scripting-engine alert, now from the Snort log	Rule 2100498

9.2 Investigation record

Phase	Analyst content
Detection	Snort signature "ET SCAN Nmap Scripting Engine User-Agent Detected" (2100498) ingested by Wazuh from the Snort alert log; first seen as the same signature at 12:14:33.
Initial analysis	Checked agent, timestamp and rule; identified the HTTP user-agent "Nmap Scripting Engine"; confirmed source 192.168.56.1 is the lab host and the destination is the author's own test endpoint; searched for related events around the time.
Evidence	Source 192.168.56.1:53122; destination 192.168.56.11:80/TCP; action allowed; category Attempted Information Leak; repeated at 12:14 and 12:32; no related exploit, malware or privilege alert on the agent (Figures 9, 10, 11).
Severity assessment	Level 4 — Medium. Asset: lab test VM. Confidence: high that it is Nmap scanning. Impact so far: none observed. Single internal source, no exploitation. Not raised to High/Critical.
Possible impact	If malicious: service enumeration, discovery of vulnerable web or network services, preparation for exploitation or lateral movement.
Containment recommendation	Verify authorisation with the lab owner. If unauthorised: isolate the source host, block it at the firewall, preserve alert and packet evidence first, and increase monitoring on the target.
Recovery recommendation	Stop the scan, review services listening on port 80, close or restrict unnecessary ones, apply patches, check for changes with FIM, and validate the endpoint after remediation.
Lessons learned	Tune the scan signature for known lab scanner IPs to cut noise, alert on repeated reconnaissance from one source, keep scan windows documented, and keep Snort rules and Wazuh decoders updated.
Conclusion	**Expected lab activity:** authorised Nmap scan from the lab host. No escalation required; continue to monitor for repeat or follow-on activity.

10. SOC Analyst Dashboard Summary (Task 8)

Metric	Actual value	Interpretation
Total alerts (24 h)	156	Overall monitoring volume
Critical alerts	12 (8%)	Immediate investigation priority
High alerts	34 (22%)	Prompt analyst review
Medium alerts	67 (43%)	Context-dependent investigation

Metric	Actual value	Interpretation
Low alerts	43 (28%)	Baseline / lower priority
Total events (24 h)	1,842	Raw events processed by rules
Rules fired	78	Breadth of detection logic triggered
Authentication events	284	Authentication activity
File integrity events	174	File change activity
Top-firing rule	ET SCAN Nmap Scripting Engine — 28 alerts	Reconnaissance is the main alert driver
Most important event	ET SCAN Nmap Scripting Engine User-Agent Detected — Rule 2100498, Level 4	Highest-interest detection (assessed as expected lab scan)
Agent status	windows-agent (001) Active, 0 disconnected, 0 never connected	Endpoint monitoring healthy
Snort-related events	23 hits for query "snort" (last 24 h); top rule 2100498	Snort events integrated into Wazuh



Figure 13 — Final SOC dashboard

Evidence interpretation: Overview with event trend, alert severity (156 alerts), top agents, top rules and top event categories.

10.1 Analyst summary

During the lab the Wazuh deployment monitored the Windows 10 test endpoint (windows-agent) and processed 1,842 events that produced 156 alerts in 24 hours: 12 critical, 34 high, 67 medium and 43 low. The main alert categories were network (812 events), web (356), authentication (284), system (216) and file integrity (174). The most important event was the Nmap scripting-engine detection (Rule 2100498, Level 4), which was triaged as expected scanning by the lab host and required no escalation. The authentication tests and the file modification test produced the expected alerts, Snort alerts were visible in Wazuh, and the agent stayed active throughout.

11. Findings, Limitations & Next Steps

11.1 Technical findings

- Wazuh provides centralised visibility into endpoint and security telemetry.
- Agent connectivity is essential; the Agents page confirms registration and Discover confirms data flow.
- Controlled test events validate that detection works end to end.
- Alert investigation needs context (user, source, time, repetition), not alert titles alone.
- Rule ID, timestamp and agent are the main investigation pivots.
- File integrity monitoring gives precise evidence (size and hash change) for host changes.
- Snort and Wazuh give complementary network and endpoint perspectives.

11.2 SOC analyst observations

Observation	SOC interpretation
High event volume (1,842 events / 156 alerts)	Requires prioritisation and filtering
Repeated authentication failures (60106)	Review source, account and timing
File modification (554)	Validate that the change was authorised
Repeated Nmap-scripting alerts	Possible reconnaissance; confirm scan windows
Network alert with endpoint evidence	Stronger correlation
Disconnected agent	Monitoring gap that must be investigated

11.3 Evidence notes and limitations

- Screenshots were captured at different points on 2 October 2026, and counters differ between captures (the first Overview, Figure 3, shows 247 alerts; the final dashboard, Figure 13, shows 156). Task 8 values use the final dashboard.
- The agent counters also differ between captures: Figure 5 and Figure 12 show one agent, while Figure 3 and Figure 13 list additional agents. Section 5 and Task 8 refer only to windows-agent (001).
- Some alerts are labelled "Suricata" and others "[snort]"; both carry signature 2100498 and the same endpoints.
- The Snort alerts analysed here come from the endpoint-side Snort log. The Week 9 Kali sensor and its custom rules (1000001, 1000002) are covered as an architecture in Section 8.4, not as a tested integration.
- Only deliberately generated test traffic was used, so no claims are made about false-positive rates on real networks.

11.4 Next steps

- Install a Wazuh Agent on the Kali Snort sensor and ingest the Week 9 custom rules as described in Section 8.4.
- Add a threshold to the ICMP rule and a Wazuh rule that raises severity on repeated reconnaissance from one source.
- Move the sensor onto its own VM so detection is independent of the traffic source.
- Forward Wazuh alerts to a ticketing or notification channel to complete the response loop.

12. Conclusion

Wazuh 4.8.1 was installed and verified, a Windows endpoint was onboarded and shown Active, and safe authentication, file and network events were each located in the dashboard. Ten alerts were recorded and one, the Nmap scripting-engine detection (rule 2100498), was investigated in full and assessed as expected lab activity. Snort events were shown alongside endpoint events in Wazuh, giving the combined network and host view that a SOC analyst relies on. Together with the Week 9 sensor, the lab now covers the full path from detection to a centralised place where alerts can be triaged, correlated and acted on.